Multi-Word Integer Operations and Types

ISO/IEC JTC1 SC22 WG21 P0104R0 - 2015-09-27

Lawrence Crowl, Lawrence@Crowl.org

Introduction

When the largest integer type is too small for an application domain, one must use a larger type.

Multi-word integer operations are a necessary component of many higher-level types, such as those that appear later in the document.

Problem

The work for multi-word integer operations is tedious. For division it is also complicated. It is better to do that work once.

Solution

We propose a template for multi-word integer types that behave like built-in types. While built-in types often have insecure semantics, they are also efficient and well understood. Some programmers will prefer to use these types directly, but we also expect these types to be used as implementation tools for higher-level types.

We also propose a set of multi-word integer operations based on pointers and counts. This low-level interface is intended as an implementation tool for higher level operations and types, including those mentioned above. These operations can be built upon the wide operations presented in P0103r0 Overflow-Detecting and Double-Wide Arithmetic Operations.

Within this document, we use `S` to refer to a signed word and `U` to refer to an unsigned word.

Definition of Word

A word is the type provided by `LARGEST_SINGLE_WIDE_ALL` and defined in P0103r0 Overflow-Detecting and Double-Wide Arithmetic Operations.

Multi-Word Types

We obtain a multi-word integer type with one of the following templates.

```
template<int words> multi_int;
template<int words> multi_uint;
```

The `multi_int` type uses a two's complement representation.

The following multi-word operators take two multi-word arguments of arbitrary size and sign. The result type is a promoted type as specified for built-in integers.

```
~ * / % + - < > <= >= & | ^
= *= /= %= += -= < > <= >= &= |= ^=
```

The following multi-word operators take a left-hand multi-word argument (of arbitrary size and sign) and a right-hand integer argument. The result type is the type of the left-hand argument.

```
<< >> <<= >>=
```

All types implicitly convert to `bool` via the normal rules.

Functions return by value. Programmers should consider the composite operation and assignment operators.

Subarray by Word Operations

We provide the following operations. These operations are not intended to provide complete multi-word operations, but rather to handle subarrays with uniform operations. Higher-level operations then compose these operations into a complete operation.

`U unsigned_subarray_addin_word U* multiplicand, int length, U addend );`

> Add the word `addend` to the `multiplicand` of length `length`, leaving the result in the `multiplicand`. The return value is any carry out from the `accumulator`.

`U unsigned_subarray_add_word U* summand, const U* augend, int length, U addend );`

> Add the `addend` to the `augend` of length `length` writing the result to the `summand`, which is also of length `length`. The return value is any carry out from the `summand`.

`U unsigned_subarray_mulin_word U* product, int length, U multiplier );`

> Multiply the `product` of length `length` by the `multiplier`, leaving the result in the `product`. The return value is any carry out from the `product`.

`U unsigned_subarray_mul_word U* product, U* multiplicand, int length, U multiplier );`

> Multiply the `multiplicand` of length `length` by the `multiplier` writing the result to the `product`, which is also of length `length`. The return value is any carry out from the `product`.

`U unsigned_subarray_accmul_word U* accumulator, U* multiplicand, int length, U multiplier );`

> Multiply the `multiplicand` of length `length` by the `multiplier` adding the result to the `accumulator`, which is also of length `length`. The return value is any carry out from the `accumulator`.

For each of the two add operations above, there is a corresponding subtract operation.

For each of the seven operations above (add+sub+mul), there is a corresponding signed operation. The primary difference between the two is sign extension.

Subarray by Subarray Operations

For each of the fourteen operations in the section above, there is a corresponding operation where the 'right-hand' argument is a pointer to a subarray, which is also of length `length`.

Open Issues

There are as yet no definitions for subarray division.